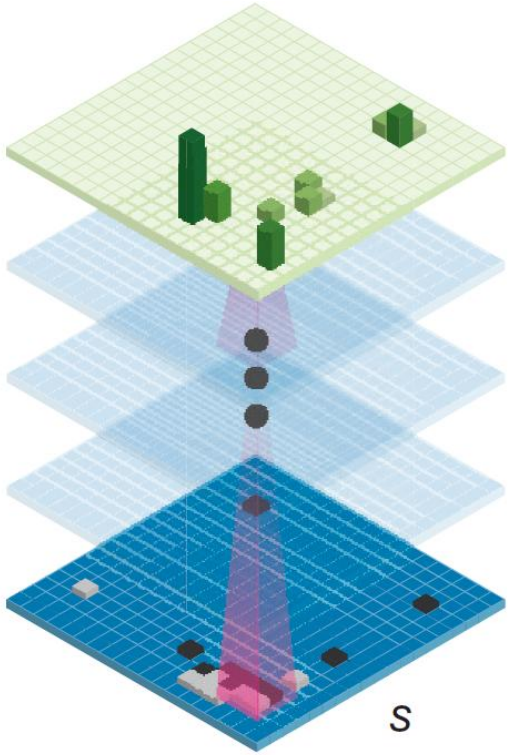


第1章 绪论

目录



第一节 引言

学习目的

课程内容

第二节 基本概念

数据与数据结构

数据类型与抽象数据类型

第三节 算法

算法的概念

算法描述

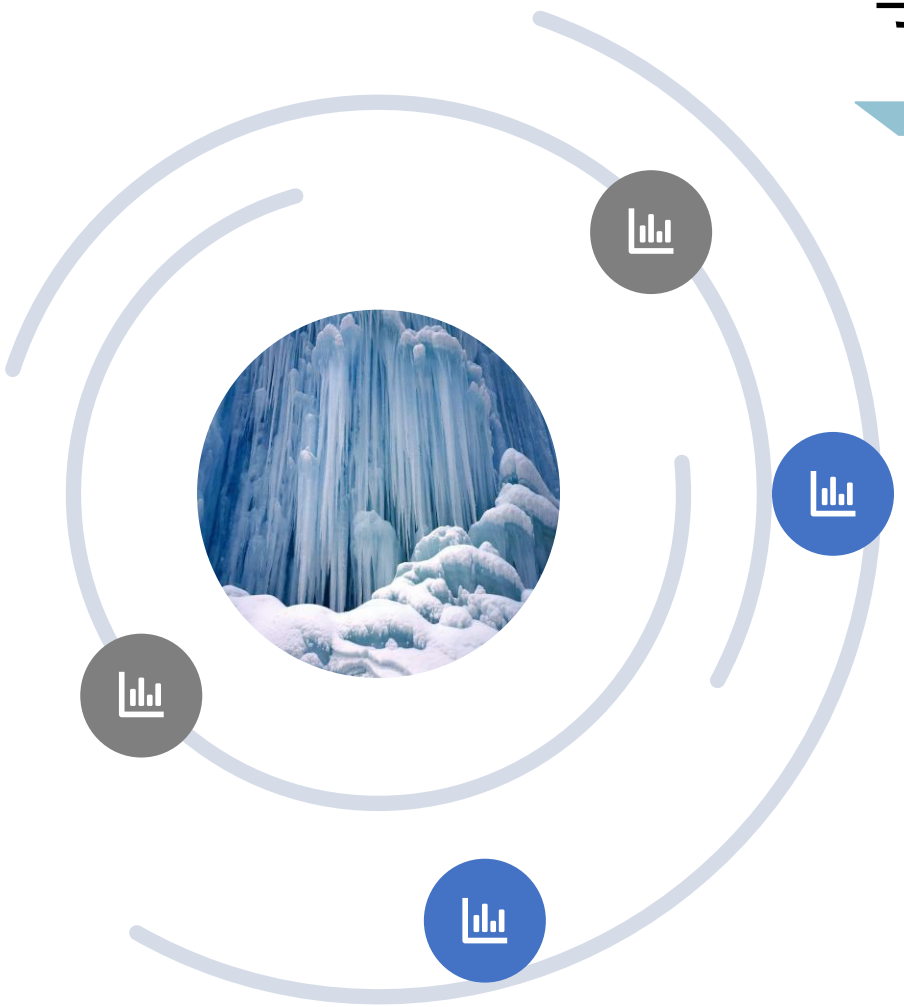
算法分析

第四节

小结

第一部分 引言

学习目的



学会使用计算机解决实际的应用问题

从具体问题中抽象出适当的数学模型： 分析问题，提取操作的对象，找出操作对象之间的逻辑关系，给出相应的数学模型。

设计解决此数学模型的算法。

编程、运行、调试，得出结果。

课程内容

过程	方面	
	数据表示	数据处理
抽象	逻辑结构	基本运算
实现	储存结构	算法
评价	不同数据结构比较和算法性能分析	

第二部分　基本概念

数据与数据结构

数据：数据(Data)是能够被计算机程序识别、存储、加工和处理的描述客观事物的数字等符号集合的总称。

数据项：数据项(Data Item)是具有独立含义的、数据不可分割的最小标识单位，是数据元素的组成部分，也可称为字段和域。

数据元素：数据元素(Data Element)是数据的基本单位，又可称为元素、结点、顶点和记录，是一个数据整体中可以标识和访问的数据单元。



数据与数据结构

数据对象：数据对象(Data Object)是性质相同的数据元素的集合，也叫数据元素类，是数据的一个子集，数据元素是数据对象的一个实例。



数据结构：数据结构(Data Structure)是相互之间存在着一种或者多种关系的数据元素的集合，数据结构概念包含3个方面的内容，即数据的逻辑结构、数据的存储结构和数据的操作，只有3个方面的内容相同才能称为完全相同的数据结构。

数据与数据结构

数据的逻辑结构

数据的逻辑结构是指数据元素之间存在的逻辑关系，由数据元素的集合和定义在此集合上的关系组成。

两要素：数据元素的集合、关系的集合。

数据逻辑结构分为以下四类

01

集合：集合中元素的关系极为松散，关系为“属于同一个集合”。

02

线性结构：线性结构是数据元素中具有线性关系的数据结构，线性结构中的结点存在“一对一”的关系。

03

树形结构：树形结构是数据元素之间具有层次关系的一种非线性结构，树形结构中的结点存在“一对多”的关系。

04

图形结构：图形结构也是一种非线性结构，图形结构中的结点存在“多对多”的关系。

数据与数据结构

数据的存储结构



逻辑结构在计算机中的存储表示或实现叫数据的存储结构，也叫物理结构。数据的逻辑结构从逻辑关系角度观察数据，与数据的存储无关，是独立于计算机的；而数据的存储结构是逻辑结构在计算机中的实现，依赖于计算机。

数据存储结构分为以下四类

01

顺序存储结构：顺序存储结构在连续的存储单元中存放数据元素，元素的物理存储次序和逻辑次序一致，即物理位置相邻的元素在逻辑上也相邻，每个元素与其前驱元素和后继元素的存储位置相邻，数据元素的物理存储结构体现它们之间的逻辑关系。

02

链式存储结构：链式存储结构使用地址分散的存储单元存放数据元素，逻辑上相邻的数据元素的物理位置不一定相邻，数据元素间的逻辑关系通常由附加的指针表示，指针记录前驱元素和后继元素的存储地址。

03

索引存储结构：索引存储结构在存储数据元素的基础上增加索引表。

04

散列存储结构：散列存储结构也叫哈希存储结构，数据元素的具体存储地址根据该数据元素的关键字值通过散列函数直接计算出来。

数据与数据结构

数据操作



数据操作是指对数据结构中的数据元素进行运算或处理。数据操作定义在数据的逻辑结构上，每种逻辑结构都需要一组对其数据元素进行处理以实现特定功能的操作，如插入、删除、更新等。数据操作的实现依赖于数据的存储结构。

常用的数据操作：

创建操作、插入操作、删除操作、查找操作、修改操作、遍历操作、销毁操作

数据类型

--数据类型

数据类型(Data Type)是一组性质相同的值的集合和定义在此集合上的一组操作的总称。在用高级程序语言编写的程序中必须对程序中出现的每个变量、常量明确说明它们所属的数据类型。



高级程序设计语言通常预定义基本数据类型和构造数据类型。

- 基本数据类型：只能作为一个整体来进行处理不可分解的数据类型。
- 构造数据类型：使用已有的基本数据类型和已定义的构造数据类型通过一定的语法规则组织起来的数据类型。

数据类型

--数据抽象

数据抽象是指“定义和实现相分离”，即将一个类型的数据及其上的操作的逻辑含义和具体实现相分离，只考虑执行什么操作(做什么)，而不考虑怎样实现这些操作(怎样做)。



- 数据抽象是一种信息隐蔽技术，可利用数据抽象研究复杂对象，忽略次要和实现细节，抽象出本质特征，抽象层次越高，复用程度越高。
- 数据抽象是通过抽象数据类型来实现的。

抽象数据类型

--抽象数据类型

抽象数据类型(Abstract Data Type，ADT)是从问题的数学模型中抽象出来的逻辑结构及定义在逻辑结构上的一组操作，仅描述了数据的特性和数据操作的语法规则，隐藏了数据的存储结构和操作的实现细节。



- 抽象数据类型是实现软件模块化设计思想的重要手段，一个抽象数据类型是描述一种特定功能的基本模块，由各种基本模块可组织和构造起来一个大型的软件系统。
- 在一般的面向对象语言中，抽象数据类型通常可以采用抽象类或接口的方式进行描述。

第三节 算法

算法的概念

算法是有穷规则的集合，其规则确定一个解决某一特定类型问题的指令序列，其中每一条指令表示计算机的一个或者多个操作。

算法必须满足以下5个特性：

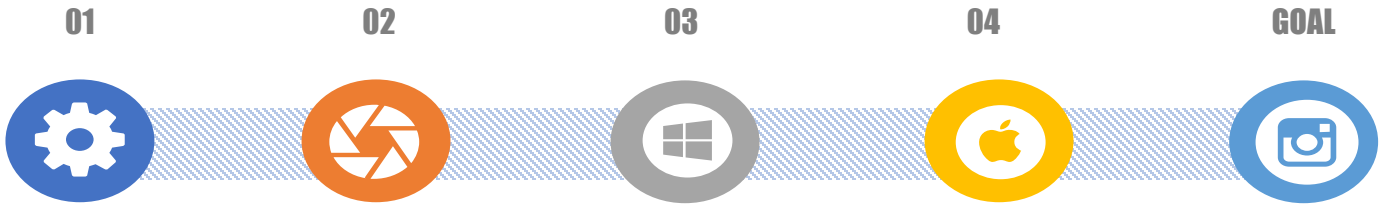
- 有穷性：对于任意的合法输入值，算法必须在执行有穷步骤后结束，并且每一步都在有穷的时间内完成。
- 确定性：算法对各种情况下执行的每个操作都有确切的规定，算法的执行者和阅读者都能明确其含义和如何执行，并且在任何条件下算法都只有一条执行路径。
- 可行性：算法中的操作必须都可以通过已经实现的基本操作运算有限次实现，并且每一条指令都符合语法规则，满足语义要求，都能被确切执行。
- 有输入：输入数据是算法的处理对象，一个算法具有零个或多个输入数据，既可以由算法指定，也可以在算法执行过程中通过输入得到。
- 有输出：输出数据是算法对输入数据进行信息加工后得到的结果，输出数据和输入数据具有确定的对应关系，即算法的功能。一个算法有一个或多个输出数据。



算法的概念

算法设计的4个基本目标

- 正确性：算法应满足应用问题的需求，这是算法设计最重要、最基本的目标。
- 健壮性：算法应具有良好的容错性，可以检查错误是否出现并且对错误进行适当的处理，即使输入的数据不合适，也能避免出现不可控的结果。
- 高效率：算法的执行时间越短，时间效率越高； 算法执行时所占的存储空间越小，空间效率越高。时间效率和空间效率往往不可兼得，用户在解决实际问题时要根据实际情况权衡得失，进行高效率算法的设计。
- 可读性：算法的表达思路应清晰，层次分明，易于理解，可读性强，以便于后续对算法的使用和修改。





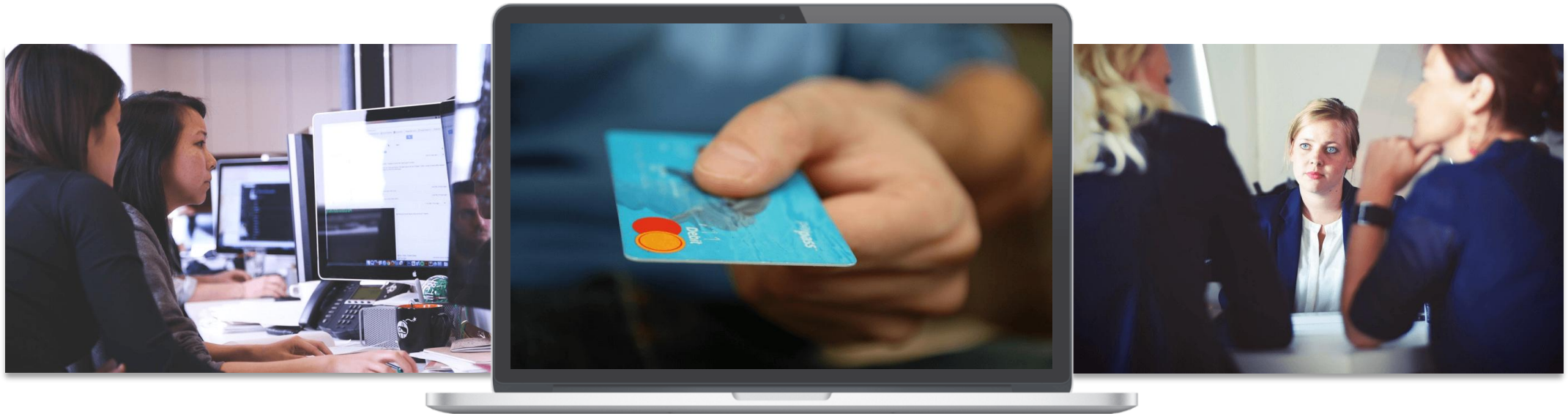
算法的概念

--算法与数据结构

算法建立在数据结构之上，对数据结构的操作需要使用算法来描述。算法设计依赖于数据的逻辑结构，算法实现依赖于数据的存储结构。



算法描述



自然语言：自然语言用中文或英文对算法进行表达，简单易懂，但缺乏严谨性。

程序设计语言：使用某种具体的程序设计语言（如Python语言）对算法进行描述。此种方式严谨，算法可直接在计算机上执行，但算法复杂、不易理解，需要借助大量的外部注释才能使用户明白。

伪代码：伪代码是介于自然语言和程序设计语言之间的算法描述语言，是将程序设计语言的语法规则用自然语言进行表示，忽略了严格的语法规则和描述细节，更易被用户理解，并且更容易转换成程序设计语言执行。

算法描述

- 例题

设计算法求两个整数的最大公约数。

01

质因数分解法

假设c为两个整数a和b的最大公约数。用数学方法求两个数的最大公约数，分别将a和b两个整数分解成若干质因数的乘积，再从中选择最大的公约数。此种方法很难用于实际计算之中，因为大数的质因数很难进行分解。

02

更相减损法

中国古代的数学经典著作《九章算术》中写道：“以少减多，更相减损，求其等也，以等数约之，即除也，其所以相减者皆等数之重叠，顾以等数约之。”其中，等数即指两数的最大公约数。

03

辗转相除法

实际上，辗转相除法就是现代版的更相减损术，使用循环实现

```
1  def gcd(a,b):
2          while b != 0 :
3                  tmp = a % b
4                  a = b
5                  b = tmp
6          return a
```

算法分析

算法分析技术主要是通过某种方法讨论算法的复杂度，评价算法的效率，以便在解决实际问题时根据实际情况和算法的优缺点对算法进行取舍。



算法的优劣主要通过算法复杂度进行衡量，复杂度的高低反映了所需计算机资源的多少。计算机资源主要包括时间资源和空间资源。因此算法的复杂度通常以时间复杂度和空间复杂度来体现。

算法分析

--算法的时间复杂度

算法的时间复杂度(Time Complexity)是指算法的执行时间随问题规模的变化而变化的趋势，反映算法执行时间的长短。



当问题的规模为n时，T(n)表示此时算法的执行时间，称为算法的时间复杂度，当n增大时，T(n)也随之增大。

假设一个算法是由n条指令序列构成的集合，算法的执行时间如下：

$$\mathrm{T(n)} = \sum_{i=1}^{n} \text{指令序列}(i)\text{的执行次数} \times \text{指令序列}(i)\text{的执行时间}$$

算法分析

--算法的时间复杂度

通常采用算法的渐进分析中的大O表示法作为算法时间复杂度的渐进度量值，称为算法的渐进时间复杂度。大O表示法是指当且仅当存在正整数c和n¬0，使得0≤T(n)≤cf(n)对所有的n≥n0成立时，算法执行时间的增长率与f(n)的增长率相同，记为T(n)=O(f(n))。

一般地，如果f(n)=aknk+ak-1nk-1+…+a1n1+a0，且ai≥0，T(n)=O(nk)，即使用大O表示法时只需保留关于数据元素个数n的多项式的最高次幂的项并去掉其系数。比如，若算法的执行时间是常数级，不依赖数据量的大小，则时间复杂度为O(1)； 若算法的执行时间与数据量为线性关系，则时间复杂度为O(n)； 对数级、平方级、立方级、指数级的时间复杂度分别为O(log2n)、O(n2)、O(n3)、O(2n)。这些函数按数量级递增排列具有下列关系：

O(1)<O(log2n)<O(n)<O(nlog2n)<O(n2)<O(n3)<O(2n)



算法分析

--算法的时间复杂度

循环语句的时间代价一般可用以下3条原则进行分析：

（1） 一个循环的时间代价=循环次数每次执行的基本指令数目

（2） 多个并列的循环的时间代价=每个循环的时间代价之和

（3） 多层嵌套循环的时间代价=每层循环的时间代价之积



算法分析

--算法的空间复杂度

算法的空间复杂度(Space Complexity)是指算法执行时所占用的额外存储空间量随问题规模的变化而变化的趋势。



执行一个算法所需要的存储空间主要包含以下两个部分。

(1) 固定空间部分： 主要包括算法的程序指令、常量、变量所占的空间，与所处理问题的规模无关。

(2) 可变空间部分： 主要包括输入的数据元素占用的空间和程序运行过程中额外的存储空间，与处理问题的规模有关。

算法分析

--算法的空间复杂度

当问题的规模为n时，S(n)表示此时算法占用的存储空间量，称为算法的空间复杂度，当n增大时S(n)也随之增大。

通常采用算法的渐进分析中的大O表示法作为算法空间复杂度的渐进度量值，称为算法的渐进空间复杂度，记作S(n)=O(f(n))，其分析与算法的渐进时间复杂度相同。



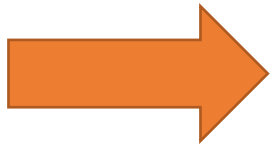
算法描述

--例题



设n为正整数，试确定下列程序段中语句“x+=1”的频度。

```
1   for i in range(n):
2           for j in range(i):
3                   x += 1
```



```
1   for i in range(n):
2           for j in range(i):
3                   for k in range(n)
4                           x += 1
```

算法描述

--例题



设n为正整数，试确定下列程序段中语句“x+=1”的频度。

```
1   i, j=0, 1
2   while i+j<=n:
3           x += 1
4           if i>j:
5                   j +=
6           else:
7                   i +=
```



```
1   x, y=91, 100
2   while y>0:
3           if(x>100):
4                   x -= 1
5                   y -= 1
6           else:
7                   x += 1
```

算法描述

--例题



设n为正整数，试确定下列程序段中语句“x+=1”的频度。

解：

（1）i为1时，j值只能取1，语句执行一次； i为2时，j可取1或2，语句执行两次； i为n时，j可取1、2、…,n，语句执行n次，所以语句频度=1+2+…+n=n(n+1)/2。

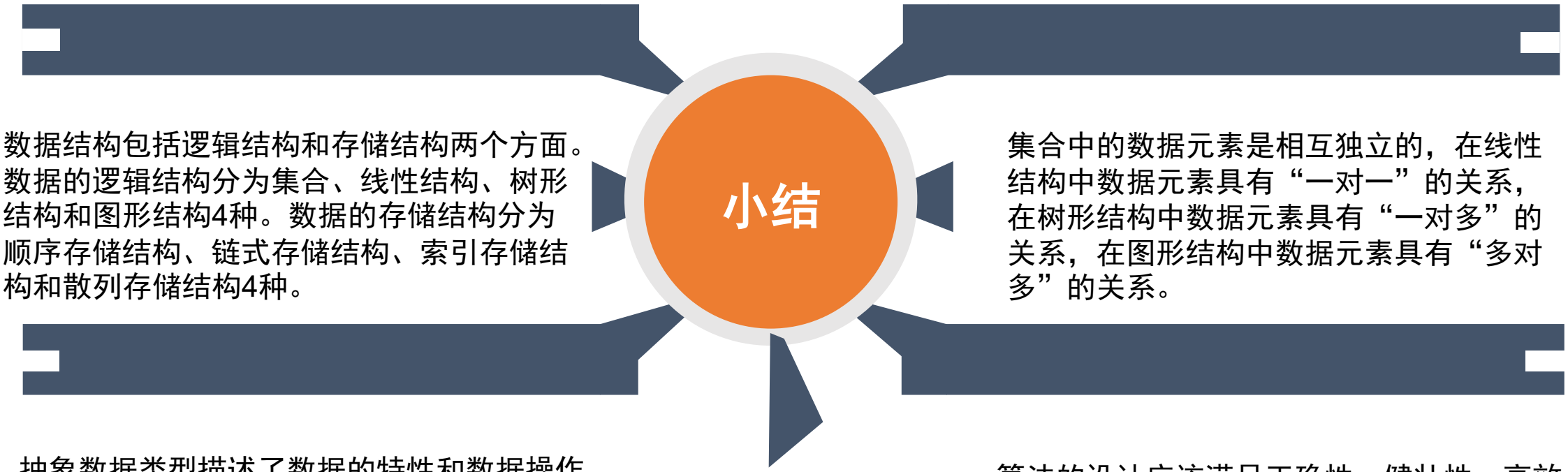
（2）i为1时，j值只能取1，k值可取1、2、…、n，语句执行n次； i为2时，j可取1或2，k值可取1、2、…、n，语句执行2n次； i为n时，j可取1、2、…、n，语句执行n×n次，所以语句频度=$n^2(n+1)/2$。

（3）i与j初始和为1，其后每循环一次i和j中有且仅有一个值增1，即i与j的和增1。由于循环条件为i+j<=n，因此循环共执行n次。所以，语句频度为n。

（4）分析y的初始值为100，当y≤0时循环结束，x++每执行10次y减小1，所以x+=1的语句频度为1000。

第四节 小结

小结



数据结构包括逻辑结构和存储结构两个方面。数据的逻辑结构分为集合、线性结构、树形结构和图形结构4种。数据的存储结构分为顺序存储结构、链式存储结构、索引存储结构和散列存储结构4种。

集合中的数据元素是相互独立的，在线性结构中数据元素具有“一对一”的关系，在树形结构中数据元素具有“一对多”的关系，在图形结构中数据元素具有“多对多”的关系。

抽象数据类型描述了数据的特性和数据操作的语法规则，隐藏了数据的存储结构和操作的实现细节。在Python语言中抽象数据类型用抽象类的定义来实现。

算法分析主要包括对时间复杂度和空间复杂度的分析，通常用大O表示法进行表示。

算法的设计应该满足正确性、健壮性、高效率和可读性4个目标。

谢谢观看